

第17章 STM32 中断应用概览

本章参考资料《STM32F4xx 中文参考手册》第十章-中断和事件、《ARM Cortex™-M4F 技术参考手册》-4.3 章节：NVIC 和 4.4 章节：SCB—4.4.5 的 AIRCR。

STM32 中断非常强大，每个外设都可以产生中断，所以中断的讲解放在哪一个外设里面去讲都不合适，这里单独抽出一章来做一个总结性的介绍，这样在其他章节涉及到中断部分的知识我们就不用费很大的篇幅去讲解，只要示意性带过即可。

本章如无特别说明，异常就是中断，中断就是异常，请不要刻意钻牛角尖较劲。

17.1 异常类型

F407 在内核水平上搭载了一个异常响应系统，支持为数众多的系统异常和外部中断。其中系统异常有 10 个，外部中断有 82 个。除了个别异常的优先级被定死外，其它异常的优先级都是可编程的。有关具体的系统异常和外部中断可在标准库文件 `stm32f4xx.h` 这个头文件查询到，在 `IRQn_Type` 这个结构体里面包含了 F4 系列全部的异常声明。

表格 17-1 F407 系统异常清单

编号	优先级	优先级类型	名称	说明	地址
	-	-	-	保留（实际存的是 MSP 地址）	0X0000 0000
	-3	固定	Reset	复位	0X0000 0004
	-2	固定	NMI	不可屏蔽中断。RCC 时钟安全系统 (CSS) 连接到 NMI 向量	0X0000 0008
	-1	固定	HardFault	所有类型的错误	0X0000 000C
	0	可编程	MemManage	存储器管理	0X0000 0010
	1	可编程	BusFault	预取指失败，存储器访问失败	0X0000 0014
	2	可编程	UsageFault	未定义的指令或非法状态	0X0000 0018
	-	-	-	保留	0X0000 001C-0X0000 002B
	3	可编程	SVCall	通过 SWI 指令调用的系统服务	0X0000 002C
	4	可编程	Debug Monitor	调试监控器	0X0000 0030
	-	-	-	保留	0X0000 0034
	5	可编程	PendSV	可挂起的系统服务	0X0000 0038
	6	可编程	SysTick	系统嘀嗒定时器	0X0000 003C

表格 17-2 F407 外部中断清单

编号	优先级	优先级类型	名称	说明	地址
0	7	可编程	-	窗口看门狗中断	0X0000 0040
1	8	可编程	PVD	连接 EXTI 线的可编程电压检测中断	0X0000 0044
2	9	可编程	TAMP_STAMP	连接 EXTI 线的入侵和时间戳中断	0X0000 0048
中间部分省略，详情请参考 STM32F4xx 中文参考手册》第十章-中断和事件-向量表部分					

79	86	可编程	CRYP	CRYP 加密全局中断	0X 0000 017C
80	87	可编程	HASH_RNG	哈希和随机数发生器全局中断	0X 0000 0180
81	88	可编程	FPU	FPU 全局中断	0X 0000 0184

17.2 NVIC 简介

在讲如何配置中断优先级之前，我们需要先了解下 NVIC。NVIC 是嵌套向量中断控制器，控制着整个芯片中断相关的功能，它跟内核紧密耦合，是内核里面的一个外设。但是各个芯片厂商在设计芯片的时候会对 Cortex-M4 内核里面的 NVIC 进行裁剪，把不需要的部分去掉，所以说 STM32 的 NVIC 是 Cortex-M4 的 NVIC 的一个子集。

17.2.1 NVIC 寄存器简介

在固件库中，NVIC 的结构体定义可谓是颇有远虑，给每个寄存器都预览了很多位，恐怕为的是日后扩展功能。不过 STM32F407 可用不了这么多，只是用了部分而已，具体使用了多少可参考《ARM Cortex™-M4F 技术参考手册》-4.3.11:NVIC 寄存器映射。

代码 11 NVIC 结构体定义，来自固件库头文件：core_cm4.h

```

1 typedef struct {
2     __IO uint32_t ISER[8];           // 中断使能寄存器
3     uint32_t RESERVED0[24];
4     __IO uint32_t ICER[8];           // 中断清除寄存器
5     uint32_t RESERVED1[24];
6     __IO uint32_t ISPR[8];           // 中断使能悬起寄存器
7     uint32_t RESERVED2[24];
8     __IO uint32_t ICPR[8];           // 中断清除悬起寄存器
9     uint32_t RESERVED3[24];
10    __IO uint32_t IABR[8];            // 中断有效位寄存器
11    uint32_t RESERVED4[56];
12    __IO uint8_t IP[240];             // 中断优先级寄存器(8Bit wide)
13    uint32_t RESERVED5[644];
14    __O  uint32_t STIR;               // 软件触发中断寄存器
15 } NVIC_Type;

```

在配置中断的时候我们一般只用 ISER、ICER 和 IP 这三个寄存器，ISER 用来使能中断，ICER 用来失能中断，IP 用来设置中断优先级。

17.2.2 NVIC 中断配置固件库

固件库文件 core_cm4.h 的最后，还提供了 NVIC 的一些函数，这些函数遵循 CMSI 规则，只要是 Cortex-M4 的处理器都可以使用，具体如下：

表格 17-3 符合 CMSIS 标准的 NVIC 库函数

NVIC 库函数	描述
void NVIC_EnableIRQ(IRQn_Type IRQn)	使能中断
void NVIC_DisableIRQ(IRQn_Type IRQn)	失能中断
void NVIC_SetPendingIRQ(IRQn_Type IRQn)	设置中断悬起位
void NVIC_ClearPendingIRQ(IRQn_Type IRQn)	清除中断悬起位
uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn)	获取悬起中断编号
void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)	设置中断优先级

uint32_t NVIC_GetPriority(IRQn_Type IRQn)	获取中断优先级
void NVIC_SystemReset(void)	系统复位

这些库函数我们在编程的时候用的都比较少，甚至基本都不用。在配置中断的时候我们还有更简洁的方法，请看中断编程小节。

17.3 优先级的定义

17.3.1 优先级定义

在 NVIC 有一个专门的寄存器：中断优先级寄存器 NVIC_IPRx（在 F407 中，x=0...981）用来配置外部中断的优先级，IPR 宽度为 8bit，原则上每个外部中断可配置的优先级为 0~255，数值越小，优先级越高。但是绝大多数 CM4 芯片都会精简设计，以致实际上支持的优先级数减少，在 F407 中，只使用了高 4bit，如下所示：

表格 17-4 F407 使用 4bit 表达优先级

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
用于表达优先级				未使用，读回为 0			

用于表达优先级的这 4bit，又被分组为抢占优先级和子优先级。如果有多个中断同时响应，抢占优先级高的就会抢占抢占优先级低的优先得到执行，如果抢占优先级相同，就比较子优先级。如果抢占优先级和子优先级都相同的话，就比较他们的硬件中断编号，编号越小，优先级越高。

17.3.2 优先级分组

优先级的分组由内核外设 SCB 的应用程序中断及复位控制寄存器 AIRCR 的 PRIGROUP[10:8]位决定，F407 分为了 5 组，具体如下：主优先级=抢占优先级

PRIGROUP[2:0]	中断优先级值 PRI_N[7:4]			级数	
	二进制点	主优先级位	子优先级位	主优先级	子优先级
0b 011	0b xxxx	[7:4]	None	16	None
0b 100	0b xxx.y	[7:5]	[4]	8	2
0b 101	0b xx.yy	[7:6]	[5:4]	4	4
0b 110	0b x.yyy	[7]	[6:4]	2	9
0b 111	0b .yyy	None	[7:4]	None	16

设置优先级分组可调用库函数 NVIC_PriorityGroupConfig() 实现，有关 NVIC 中断相关的库函数都在库文件 misc.c 和 misc.h 中。

代码 12 中断优先级分组库函数

```

1 /**
2  * 配置中断优先级分组：抢占优先级和子优先级
3  * 形参如下：
4  * @arg NVIC_PriorityGroup_0: 0bit for 抢占优先级
5  *                          4 bits for 子优先级
6  * @arg NVIC_PriorityGroup_1: 1 bit for 抢占优先级
7  *                          3 bits for 子优先级
8  * @arg NVIC_PriorityGroup_2: 2 bit for 抢占优先级
9  *                          2 bits for 子优先级

```

```

10  * @arg NVIC_PriorityGroup_3: 3 bit for 抢占优先级
11  *                               1 bits for 子优先级
12  * @arg NVIC_PriorityGroup_4: 4 bit for 抢占优先级
13  *                               0 bits for 子优先级
14  * @注意 如果优先级分组为 0，则抢占优先级就不存在，优先级就全部由子优先级控制
15  */
16 void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup)
17 {
18     // 设置优先级分组
19     SCB->AIRCR = AIRCR_VECTKEY_MASK | NVIC_PriorityGroup;
20 }

```

表格 17-5 优先级分组真值表

优先级分组	主优先级	子优先级	描述
NVIC_PriorityGroup_0	0	0-15	主-0bit, 子-4bit
NVIC_PriorityGroup_1	0-1	0-7	主-1bit, 子-3bit
NVIC_PriorityGroup_2	0-3	0-3	主-2bit, 子-2bit
NVIC_PriorityGroup_3	0-7	0-1	主-3bit, 子-1bit
NVIC_PriorityGroup_4	0-15	0	主-4bit, 子-0bit

17.4 中断编程

在配置每个中断的时候一般有 3 个编程要点：

- 1、使能外设某个中断，这个具体由每个外设的相关中断使能位控制。比如串口有发送完成中断，接收完成中断，这两个中断都由串口控制寄存器的相关中断使能位控制。
- 2、初始化 NVIC_InitTypeDef 结构体，配置中断优先级分组，设置抢占优先级和子优先级，使能中断请求。

代码 13 NVIC 初始化结构体

```

1 typedef struct {
2     uint8_t NVIC_IRQChannel;           // 中断源
3     uint8_t NVIC_IRQChannelPreemptionPriority; // 抢占优先级
4     uint8_t NVIC_IRQChannelSubPriority;   // 子优先级
5     FunctionalState NVIC_IRQChannelCmd;   // 中断使能或者失能
6 } NVIC_InitTypeDef;

```

有关 NVIC 初始化结构体的成员我们一一解释下：

- 1) NVIC_IRQChannel: 用来设置中断源，不同的中断中断源不一样，且不可写错，即使写错了程序不会报错，只会导致不想要中断。具体的成员配置可参考 stm32f4xx.h 头文件里面的 IRQn_Type 结构体定义，这个结构体包含了所有的中断源。

代码 14 IRQn_Type 中断源结构体

```

1 typedef enum IRQn {
2     //Cortex-M4 处理器异常编号
3     NonMaskableInt_IRQn    = -14,
4     MemoryManagement_IRQn  = -12,
5     BusFault_IRQn           = -11,
6     UsageFault_IRQn         = -10,
7     SVC_IRQn                = -5,

```

```
8   DebugMonitor_IRQn      = -4,
9   PendSV_IRQn            = -2,
10  SysTick_IRQn           = -1,
11  //STM32 外部中断编号
12  WWDG_IRQn              = 0,
13  PVD_IRQn               = 1,
14  TAMP_STAMP_IRQn        = 2,
15
16  // 限于篇幅，中间部分代码省略，具体的可查看库文件 stm32f4xx.h
17
18  CRY_P_IRQn              = 79,
19  HASH_RNG_IRQn          = 80,
20  FPU_IRQn               = 81,
21 } IRQn_Type;
```

2) NVIC_IRQChannelPreemptionPriority: 抢占优先级，具体的值要根据优先级分组来确定，具体参考表格 17-5 优先级分组真值表。

3) NVIC_IRQChannelSubPriority: 子优先级，具体的值要根据优先级分组来确定，具体参考表格 17-5 优先级分组真值表。

4) NVIC_IRQChannelCmd: 中断使能 (ENABLE) 或者失能 (DISABLE)。操作的是 NVIC_ISER 和 NVIC_ICER 这两个寄存器。

3、编写中断服务函数

在启动文件 startup_stm32f40xx.s 中我们预先为每个中断都写了一个中断服务函数，只是这些中断函数都是为空，为的只是初始化中断向量表。实际的中断服务函数都需要我们重新编写，中断服务函数我们统一写在 stm32f4xx_it.c 这个库文件中。

关于中断服务函数的函数名必须跟启动文件里面预先设置的一样，如果写错，系统就在中断向量表中找不到中断服务函数的入口，直接跳转到启动文件里面预先写好的空函数，并且在里面无限循环，实现不了中断。